

输入输出函数

- 输入：有符号整数，ReadInt，获得的数据存在eax中
- ; Receives: nothing
- ; Returns: If CF=0, the integer is valid, and EAX = binary value.
- ; If CF=1, the integer is invalid and EAX = 0.
- 输出：WriteInt，准备的打印的数据放入eax中
- ; Receives: EAX = the integer

输入输出函数

- 输入: ReadString
- ; Receives: EDX points to the input buffer,
- ; ECX contains the maximum string length
- ; Returns: EAX = size of the input string.

```
mov  edx, offset Linputarea      ; save offset in edx
mov  ecx, LMAX_DIGITS
call ReadString
mov  ecx, eax                    ; save length in CX
```

- 输出: WriteString
- ; output. Input parameter: EDX points to the string.

```
mov  edx, offset waitmsgstr
call WriteString
```

输入输出函数

- DumpMem
- ; Receives: ESI = starting offset, ECX = number of units,
- ; EBX = unit size (1=byte, 2=word, or 4=doubleword)
- ; Returns: nothing
- 例子:
- `mov esi,offset dat ;esi存储内存开始地址`
- `mov ebx,type dat ;ebx存储数据类型字量`
- `mov ecx,lengthof dat;ecx存储内存单元数量`
- `call dumpmem`

其余的输入输出函数，详细见**Irvine32.asm**

```
;-----  
ReadString proc  
;  
; Reads a string of up to 128 characters from the console  
; past the end of line, and places the characters in a buffer.  
; Strips off CR/LF from the input buffer.  
; Receives: EDX points to the input buffer,  
;          ECX contains the maximum string length  
; Returns: EAX = size of the input string.  
; Comments: Stops when Enter key (0Dh) is pressed. Because  
ReadConsole  
; places 0Dh,0Ah in the buffer, we have to use a local variable  
; to overwriting the caller's memory. After calling ReadConsole,  
; we copy the characters back to the caller's buffer.  
; Last update: 8/14/01  
;-----
```

.data

READSTRING_BUFSIZE = 128

InputTooLong BYTE "ReadString error: Cannot input string longer than 128 bytes.",0Dh,0Ah,0

localBuf BYTE (READSTRING_BUFSIZE + 2) DUP(?)

.code

pushad

CheckInit

cmp ecx,READSTRING_BUFSIZE

ja L3Err ; InputTooLong

add ecx,2 ; include CR/LF in the input

push edx

INVOKE ReadConsole,

consoleInHandle, ; console input handle

OFFSET localBuf, ; pointer to local buffer

ecx, ; max count

OFFSET bytesRead,

0

pop edx

sub bytesRead,2 ; adjust character count

jz L5 ; skip move if zero chars input

```

L1:                ; Copy from the local buffer to the caller's buffer.
                   mov ecx,bytesRead
                   mov esi,OFFSET localBuf

L2:                mov al,[esi]
                   mov [edx],al
                   inc esi
                   inc edx
                   Loop L2

L5:                mov BYTE PTR [edx],0                ; add NULL
byte
                   jmp L4

L3Err:             mov edx,OFFSET InputTooLong
                   call WriteString

L4:                popad
                   mov eax,bytesRead
                   ret

ReadString endp

```